INTERN



Scrum

Mühlespiel

2. Juni 2025

Bellini Silvio, Mikus Ryan, Sutter Christian

OST
Ostschweizer Fachhochschule
Werdenbergstrasse 4, 9471 Buchs

Änderungsnachweis

Autor: Bellini Silvio, Mikus Ryan, Sutter Christian
Erstellt am: 2. Juni 2025

Version	Änderungsgrund	Kurz-Z.	Datum
1	Erstellung der Dokumentation	01	02.06.25

Inhaltsverzeichnis

1. Einleitung ... 1
1.1 Zweck ... 1
1.2 Scrum ... 1
1.3 Team ... 1
2. Überblick Scrum ... 2
3. Meilensteine ... 3
3.1 Projektplanung ... 3
3.2 Grundgerüst und Spiellogik ... 3
3.3 KI, GUI und History ... 3
3.4 Intelligente KI und Multiplayer ... 3
3.5 Optimierungen und Tests ... 3
3.6 Release ... 3
4. Resultate ... 4
4.1 Projektplanung ... 4
4.2 Grundgerüst und Spiellogik ... 5
4.3 KI, GUI und History ... 6
4.4 Intelligente KI und Multiplayer ... 7
4.5 Optimierungen und Tests ... 8
4.6 Release ... 9
5. Erfahrungen ... 10
6. Abbildungsverzeichnis ... 11

1. Einleitung

1.1 Zweck

In diesem Dokument soll der gesamte Planungsprozess unter Verwendung von Scrum beschrieben werden. Damit soll transparent gemacht werden, wie das Projekt geplant und durchgeführt wurde.

1.2 Scrum

Scrum eignet sich besonders gut für Softwareentwicklungsprojekte, da es eine inkrementelle Vorgehensweise erlaubt, bei der in kurzen Zeitabständen funktionsfähige Teilergebnisse geliefert werden können. Das Projekt wurde in unterschiedliche Meilensteine unterteilt, wovon für jeden vor Start des Meilensteins Aufgaben definiert wurden.

1.3 Team

Das Team bestand aus drei Personen, welche alle die Funktion eines Entwicklers hatten. Zusätzlich wurden zu Beginn des Projekts einzelne Aufgabenbereiche zugeteilt, sowie Schwerpunkte der zu entwickelnden Software.

Ryan Mikus hatte die Rolle des «Product Owners» und war zuständig für das Einhalten der Vorgaben, sowie die Dokumentation. Er kümmerte sich hauptsächlich um die Implementierung des KI-Gegners und dessen Schwierigkeitsgrade, welche unterschiedliche Taktiken verwenden.

Silvio Bellini hatte die Rolle des «Scrum Masters» und war zuständig für die Planung und Koordination der Aufgaben der einzelnen Meilensteine. Zusätzlich kümmerte er sich um das Notieren der Resultate der Meilensteinreviews. Er kümmerte sich hauptsächlich um die Spiellogik, sowie das Abspeichern und Laden von Spielständen.

Christian Sutter hatte die Rolle des Teammitglieds und somit keine zusätzliche Aufgabe. Er kümmerte sich hauptsächlich um die Darstellung und Benutzerinteraktion, sowie um die History, mit welcher die gemachten Züge rückgängig gemacht oder wiederholt werden können.

2. Überblick Scrum

Für die Planung wurde direkt auf GitLab mit einem Issueboard gearbeitet. Zunächst wurden die Meilensteine definiert. Jeweils vor Beginn des nächsten Meilensteins wurden die Issues dazu erstellt. Diese enthielten einen Titel, eine detaillierte Beschreibung der Aufgabe, Tags zur Klassifizierung und den zugehörigen Meilenstein.

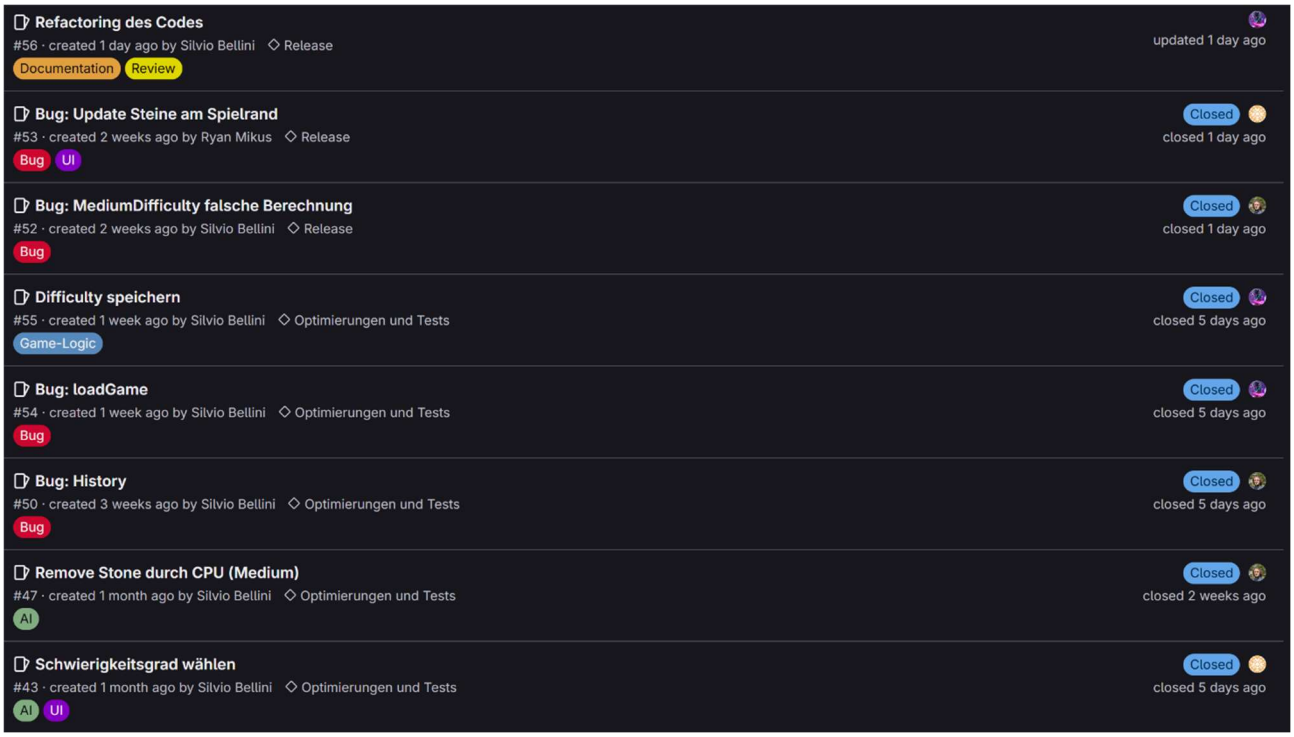


Abbildung 1: Issues

Es wurde in Sprints von jeweils einer Woche gearbeitet, welche am Montag jeder Woche besprochen wurden. Darin wurden der bisherige Stand, Schwierigkeiten und Anregungen oder Ideen diskutiert. Bei Ende eines Meilensteins wurde dieser zusätzlich besprochen, indem positive und negative Erfahrungen, sowie zu verschiebende Issues festgehalten wurden. Falls nötig wurde sich auch unter der Woche über einzelne Issues ausgetauscht.



Abbildung 2: Meilensteine

3. Meilensteine

Das Projekt wurde insgesamt in sechs Meilensteine unterteilt. Diese dauerten jeweils zwei Wochen, mit Ausnahme des letzten Meilensteins, für welchen noch eine Woche eingeplant wurde.

3.1 Projektplanung

Der erste Meilenstein diente der Projektplanung. Hier sollten unterschiedliche Modelle und Dokumente erzeugt werden, darunter ein Zeitplan, das Domänenmodell oder das Klassendiagramm. Ausserdem sollte das Projekt mit GitLab aufgesetzt und sichergestellt werden, dass alle Teammitglieder Zugriff darauf und keine Komplikationen haben.

3.2 Grundgerüst und Spiellogik

Beim zweiten Meilenstein sollte das Grundgerüst des Programms gemäss Klassendiagramm aufgesetzt werden. Zusätzlich sollte bereits der Grossteil der Spiellogik implementiert sein, beispielsweise die Datenstruktur des Spielbretts, das Einhalten der Spielregeln oder die Erkennung von Mühlen. Dafür sollte noch keine GUI verwendet werden, sondern lediglich mit Konsolenausgaben getestet werden.

3.3 KI, GUI und History

Beim dritten Meilenstein sollten bereits etwas komplexere Teile des Codes implementiert werden. Dazu gehörte zunächst eine simple KI, welche autonom zufällige Züge machen können sollte. Ausserdem sollten nun das Spielbrett, sowie Spielzüge visualisiert werden. Zusätzlich sollte die Interaktion des Spielers mit dem GUI ermöglicht werden. Die Grundstruktur für die History, mit welcher Züge rückgängig gemacht oder wiederholt werden können, sollte ebenfalls bereits implementiert werden.

3.4 Intelligente KI und Multiplayer

Nachdem das Spiel bereits spielbar sein sollte, sollten ein weiterer Schwierigkeitsgrad der KI und die Möglichkeit des Spielens zu zweit hinzugefügt werden. Ausserdem sollte bereits umfangreich, eventuell sogar bereits von Dritten, getestet werden.

3.5 Optimierungen und Tests

Bei diesem Meilenstein sollte das Spiel bereits grösstenteils fertig sein. Dadurch könnte die GUI etwas verbessert, eventuell mit Animationen oder Sound, oder die KI mit einem weiteren Schwierigkeitsgrad optimiert werden. Es sollten Testbenches aufgesetzt werden, mit welchen das Programm automatisch getestet werden kann. Ausserdem sollte die Dokumentation für die Abgabe bereits vorbereitet werden.

3.6 Release

Der letzte Meilenstein sollte lediglich dem Abschluss der Dokumentation und der Tests dienen. Der Rest sollte als Pufferzeit verwendet werden können, falls unerwartete Probleme auftreten würden.

4. Resultate

4.1 Projektplanung

Erreicht

Das Projekt konnte in GitLab aufsetzt werden. Zusätzlich wurden der Zeitplan und das Domänenmodell erstellt.

Review

Positives	Negatives	Verschobene Issues
Arbeitsteilung hat gut funktioniert	Mit etwas Verspätung begonnen	Klassendiagramm
Theorie aus dem Unterricht anwenden können	Reihenfolge der Schritte nicht optimal durchgeführt (Dömanenmodell vor OOA)	Objektorientierte Analyse
Projekt und Git aufgesetzt ohne Schwierigkeiten	Integration von Visualisierung in GitLab hat nicht auf Anhieb funktioniert	Pflichtenheft
Git-Projekt einfach konvertieren können in gemeinsames GitLab (Gruppe 2)	Einige Issues nicht fertiggestellt vor Ende des Meilensteins	
Teameinteilung schnell und ohne Probleme		
Gutes Tool zur Visualisierung von Diagrammen gefunden		

4.2 Grundgerüst und Spiellogik

Erreicht

Die aus dem vorherigen Meilenstein verschobenen Aufgaben konnten direkt nachgeholt werden. Somit wurden das Klassendiagramm und Pflichtenheft erstellt und eine Objektorientierte Analyse durchgeführt, wo unterschiedliche Use-Cases erzeugt wurden.

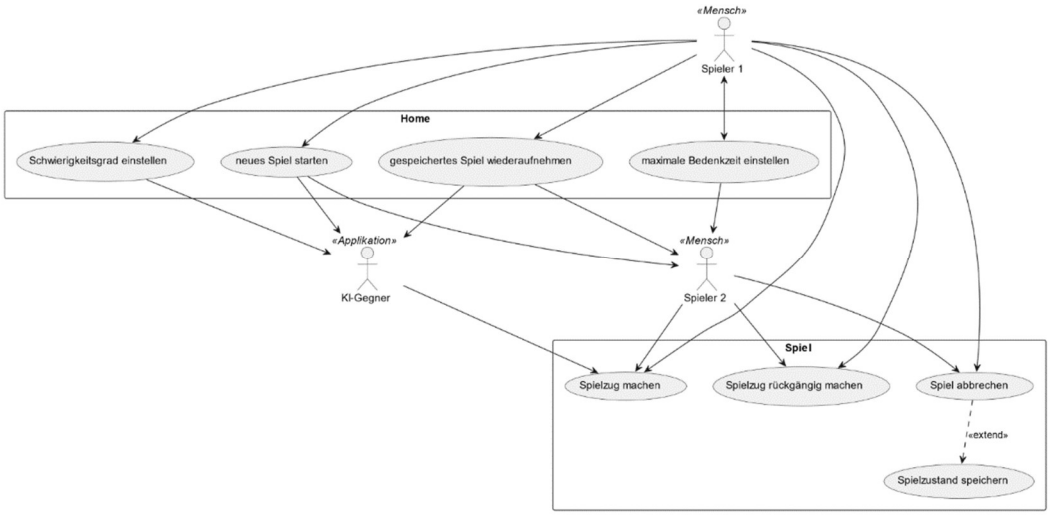


Abbildung 3: Use-Case-Diagramm

Anschliessend konnte sich um den neuen Meilenstein gekümmert werden. Anhand des Klassendiagramms liessen sich die Klassen schnell aufsetzen. Dann wurde zunächst die Datenstruktur des Spielfelds aufgesetzt, eine Hashmap bestehend aus Identifikationsnummern und den dazugehörigen Knotenpunkten. Durch die Nummerierung konnten die Verbindungen zwischen den Knotenpunkten hartkodiert werden. Als nächstes konnten die Bedingungen für das Spielende, die unterschiedlichen Phasen und das Erkennen von Mühlen implementiert und bereits getestet werden. Die GUI wurde bereits vorgezogen, auf Kosten eines Teils der Spiellogik.

Review

Positives	Negatives	Verschobene Issues
GUI konnte bereits vorbereitet werden	Wenig Zeit am Projekt zu arbeiten	Züge validieren
Grundlogik konnte ohne grössere Schwierigkeiten implementiert werden	Die Logik konnte noch nicht getestet werden	GameOverPhase implementieren
Klassendiagramm stellte gutes Grundgerüst dar, es musste nur wenig erweitert werden	Ein Teil der Logik konnte nicht komplett implementiert werden, sondern wurde vereinzelt in anderen Klassen begonnen	Stein entfernen
In den Reviews konnten einige Logikfehler frühzeitig erkannt und eliminiert werden		

4.3 KI, GUI und History

Erreicht

Auch hier wurden zunächst die verschobenen Issues des letzten Meilensteins vorgezogen. Somit konnte das Validieren von Spielzügen, Entfernen von Steinen und die GameOverPhase implementiert werden. Als nächstes konnte der simple KI-Gegner, welcher zufällige Spielzüge ausführt, aufgesetzt werden.

Parallel wurde die GUI erweitert mit unterschiedlichen Fenstern, den jeweiligen Buttonfunktionen oder das aktualisieren bei einer Änderung. Dabei wurden auch einige Bugs erkannt und gleich behoben, wie das Bewegen fremder Steine, bilden von Mühlen unabhängig der Steinfarben oder herausziehen eines neuen Spielsteins aus einem bereits platzierten.

Zusätzlich konnte die Grundstruktur für die History implementiert werden, also die Logik für das rückgängig machen oder wiederholen, sowie die Datenstruktur, welche die Informationen dazu enthält. Dabei handelte es sich um den Move, welcher die beiden Spieler, den Zustand des Spielbretts und einzelne Variablen, wie die Anzahl Spielzüge ohne Mühle, enthielt.

Review

Positives	Negatives	Verschobene Issues
Individuelles arbeiten auf separaten Branches	Spielfluss muss überarbeitet werden (momentan steuert GUIBoard den Ablauf statt des GameManagers)	Simpler KI-Gegner (Noch nicht reviewed)
Zusammenführen der Branches ohne grosse Schwierigkeiten	Schwierigkeiten beim Implementieren der KI	GUI updaten (Funktioniert, aber Move von KI muss noch hinzugefügt werden)
Einige Bugs erkannt und schnell gefixt	Code Review konnte nicht durchgeführt werden --> Spielfluss stattdessen diskutiert	Funktionalität Spielmenu Buttons (Funktioniert aber Save und Load game muss noch hinzugefügt werden)
GUI funktioniert gut und geplante Funktionalitäten sind implementiert (allerdings noch nicht vollständig getestet)	History funktioniert nicht wie geplant und Ursache ist noch unklar	
Spielfluss konnte in Besprechung gemeinsam ausführlich geplant werden	Schlechte Fehlerbehandlung und unvollständig getestet	
JavaDoc überarbeitet und funktionierende Funktionalitäten gut dokumentiert		

4.4 Intelligente KI und Multiplayer

Erreicht

Zunächst konnten die verbleibenden Aufgaben des letzten Meilensteins direkt besprochen und somit abgeschlossen werden. Anschliessend wurde der nächste Schwierigkeitsgrad der KI implementiert. Da die Logik des Mehrspielers bereits existierte, musste dieser nicht extra implementiert werden. Stattdessen wurde die GUI dafür erweitert.

Als nächstes wurde das zurücksetzen des Spielbretts ermöglicht, beispielsweise beim Beenden oder Neustarten eines Spiels. Ausserdem befand sich die Kontrolle des Spielflusses ursprünglich auf der GUI, was nicht dem Konzept der verwendeten Entwurfsmuster entsprach und durch Unachtsamkeit entstanden war. Aus diesem Grund wurde der Spielfluss in diesem Meilenstein auf den Controller, also den GameManager verschoben.

Zusätzlich konnten wiederum einige Bugs erkannt und direkt behoben werden, wie der frühzeitige Spielerwechsel oder die Ausgabe von Fehlern beim Starten des Einzelspielers.

Besonders die History bereitete grosse Schwierigkeiten, trotz viel Zeitaufwand konnte sie nicht funktionsfähig gemacht werden.

Review

Positives	Negatives	Verschobene Issues
Ausgabe des Spielflusses über LOGs und im GUI	History funktioniert immer noch nicht zuverlässig und viel Zeit wurde investiert	Spiel speichern
Saubere Aufteilung der Branches	Von History abhängige Issues konnten nicht behandelt werden	Spiel laden
Einige hartnäckige Bugs gefixt	Wenig Zeit für Reviews zu den Issues	Schwierigkeitsgrad wählen
Spielfluss auf GameManager verschoben	Spiel kann in der Setzphase enden, wird aber nicht direkt beendet --> Phasenwechsel muss optimiert werden	Remove stone durch CPU (Medium)
KI führt fehlerfrei Züge aus	Code-Struktur beginnt unübersichtlich zu werden, eventuell ist die klare Trennung der Zuständigkeiten nicht mehr gewährleistet --> Code Review wäre nötig	
Viel einzeln getestet	Tests waren ineffizient, eigene Testfunktionen könnten geschrieben werden	

4.5 Optimierungen und Tests

Erreicht

Aufgrund der Schwierigkeiten mit der History mussten einige Aufgaben des letzten Meilensteins nach hinten verschoben werden. Nachdem diese funktionsfähig gemacht wurde, konnten die Issues, also das Speichern und Laden eines Spielstandes oder die Wahl des Schwierigkeitsgrades, erfolgreich nachgeholt werden. Ansonsten wurde viel getestet und einige kleinere Bugs behoben. Allerdings konnte nicht mit Testbenches getestet werden, da diese mit der vorhandenen Struktur nicht mehr so einfach hinzugefügt werden konnten.

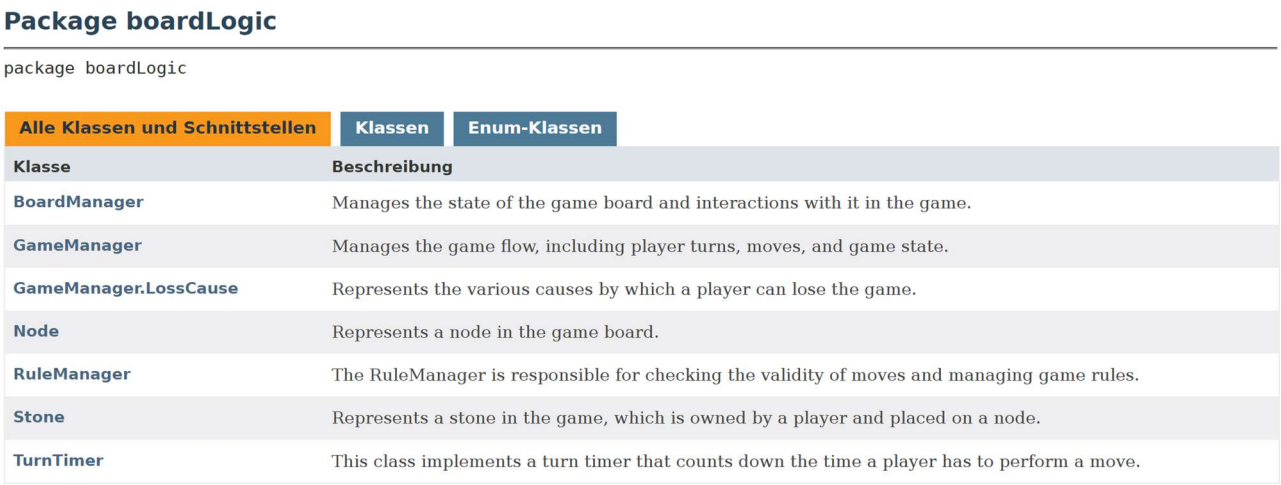
Review

Positives	Negatives	Verschobene Issues
HISTORY FUNKTIONIERT	KI führt nicht immer den Zug aus, der erwartet wird	Bug: Update Steine am Spielrand
KI hat drei Schwierigkeitsgrade, die merklich unterschiedlich sind	Bugs wurden von einer Person gefixt, führten aber bei einer anderen zu Problemen -> Aneinander vorbei "geredet"	Bug: MediumDifficulty falsche Berechnung
Mergen von mehreren Branches nacheinander hat gut funktioniert, Konflikte konnten sauber gelöst werden	Optimierung und Dokumentation konnten nicht so ausführlich behandelt werden, wie für diesen Meilenstein geplant	
Unkompliziertes Einbinden einer externen Library zum extrahieren in ein JSON-File	Metriken zeigten, dass einige Klassen noch viel zu komplex und lang sind	
Spiel ist fast komplett bugfrei und alles was angefangen wurde, konnte zusammengeführt werden	Testen wurde jeweils einzeln durchgeführt, indem das Spiel gespielt wurde. Es wurden aber keine Testbenches verwendet.	

4.6 Release

Erreicht

Zunächst wurde sich um das Refactoring des Codes gekümmert, da die Metriken deutlich aufzeigten, dass einige Klassen zu komplex sind. Daher wurden die grossen und komplexen Klassen in kleinere unterteilt oder Teile in eigene Methoden ausgelagert. Ausserdem wurde der komplette Code nochmals angeschaut und am Ende für das JavaDoc kommentiert. Danach fielen noch einige Bugs auf, welche behoben werden konnten. Zu guter Letzt wurde die Dokumentation noch aufgearbeitet und die Abgabe vorbereitet, indem das JavaDoc generiert wurde.



Package boardLogic

package boardLogic

Alle Klassen und Schnittstellen | Klassen | Enum-Klassen

Klasse	Beschreibung
BoardManager	Manages the state of the game board and interactions with it in the game.
GameManager	Manages the game flow, including player turns, moves, and game state.
GameManager.LossCause	Represents the various causes by which a player can lose the game.
Node	Represents a node in the game board.
RuleManager	The RuleManager is responsible for checking the validity of moves and managing game rules.
Stone	Represents a stone in the game, which is owned by a player and placed on a node.
TurnTimer	This class implements a turn timer that counts down the time a player has to perform a move.

Abbildung 4: JavaDoc

Review

Positives	Negatives	Verschobene Issues
Der komplette Sourcecode wurde kommentiert.	Die Klassen hätten schon zu Beginn in Packages geplant werden sollen.	
Die komplexen Klassen konnten deutlich simplifiziert werden.	Die Klassen hätten regelmässiger gewartet werden sollen.	
Es konnten keine offensichtlichen Bugs mehr festgestellt werden.	Es hätte darauf geachtet werden sollen, dass einfach mit UnitTests getestet hätte werden können. Wir mussten während des Projekts sehr viel Mühle spielen.	
Die Dokumentation konnte gut aufgeteilt und ausführlich beschrieben werden.		
Das JavaDoc konnte einfach generiert werden und sieht vollständig aus.		

5. Erfahrungen

Das Projekt war sehr interessant und lehrreich. Wir haben gelernt, wie man sich im Team mit Hilfe von Scrum organisieren kann und konnten Erfahrung in der Verwendung von Git sammeln. Gerade parallel zum Unterricht konnten viele Themen die behandelt wurden, direkt angewendet werden. Das Projekt gab einen Einblick darin, wie Projekte in der zukünftigen Arbeitsumgebung von Statten gehen könnten. Wir versuchten, Industriestandards und «Best practices», beispielsweise beim JavaDoc, möglichst gut und vollständig anzuwenden.

Es verlief nicht alles reibungslos. Besonders die History bereitete uns Schwierigkeiten, da zunächst die kopierten und wiederhergestellten Objekte Referenzen auf die alten Objekte aufwiesen. Daher mussten davon tiefe Kopien erzeugt werden. Trotzdem funktionierte sie nicht einwandfrei und wurde daher nach und nach verbessert und gefixt. Sie bereitete uns bis in den letzten Meilenstein Probleme, konnte da aber endlich soweit verbessert werden, dass keine Bugs mehr ausgemacht werden konnten.

Auch die Struktur selbst stellte einige Stolpersteine bereit. Es wurde mit dem MVC-Entwurfsmuster gearbeitet, welches für sich bereits einige Aufmerksamkeit bei der Planung der Architektur benötigt. Während der Entwicklung wurden die Grenzen des Musters nach und nach verwässert, was anschliessend mit einem ausführlichen Refactoring wieder behoben werden musste, denn der Spielfluss wurde immer mehr von der View direkt übernommen, statt von einem separaten Controller.

Der Fokus auf die Sicherstellung der folgenden Funktionsweise führte auch dazu, dass einige Klassen zu «Monster-Klassen» anwuchsen. Diese mussten ebenfalls in einem gemeinsamen Refactoring in mehrere Klassen oder Hilfsmethoden aufgeteilt werden. Insgesamt lässt sich somit sagen, dass die Architektur zu Beginn gut ausgearbeitet und durchdacht gewesen war. Doch es hätte während der Entwicklungsphase mehr darauf geachtet werden sollen, dass die Zuständigkeiten der Klassen klar getrennt bleiben.

Das Teamwork hat über das gesamte Projekt gut funktioniert. Alle waren motiviert und arbeiteten ständig an den ihnen zugeteilten Aufgaben. Es wurde regelmässig besprochen, wie weit die einzelnen Aufgaben sind und wo Schwierigkeiten liegen. Wenn dies der Fall war, wurde über Lösungsvorschläge diskutiert und so konnten Probleme schnelle behoben werden.

6. Abbildungsverzeichnis

Abbildung 1: Issues 2
Abbildung 2: Meilensteine 2
Abbildung 3: Use-Case-Diagramm 5
Abbildung 4: JavaDoc 9